

Digital Design and Computer Architecture

RISC-V Edition



表 B.4 寄存器名称与编号 Na

我	注册 数字使用	
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporary registers
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / Return val用户
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporary registers

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

funct4 rd/rs1 rs2 op CR类型					
funct3	imm	rd/rs1		imm	op
funct3	imm	rs1'		imm	rs2'
funct6		rd/rs1'		funct2	rs2'
funct3	imm	rs1'		imm	op
funct3	imm	funct	rd/rs1'	imm	op
funct3	imm				op
funct3	imm			rs2	op
funct3	imm			rd'	op
funct3	imm		rs1'	imm	rd'
3 bits		3 bits		2 bits	

Figure B.2 RISC-V compressed (16-bit) instruction formats

Table B.5 RVM: RISC-V乘法法和除法指令

操作码	funct3	funct7	类型	指令描述	操作	0110011 (51) 000 0000001 R
				mul rd, rs1, rs2 乘法		rd = (rs1 * rs2) _{31:0}
0110011 (51)	001	0000001	R	mulh rd, rs1, rs2	multiply high signed signed	rd = (rs1 * rs2) _{63:32}
0110011 (51)	010	0000001	R	mulhsu rd, rs1, rs2	multiply high signed unsigned	rd = (rs1 * rs2) _{63:32}
0110011 (51)	011	0000001	R	mulhu rd, rs1, rs2	multiply high unsigned unsigned	rd = (rs1 * rs2) _{63:32}
0110011 (51)	100	0000001	R	div rd, rs1, rs2	divide (signed)	rd = rs1 / rs2
0110011 (51)	101	0000001	R	divu rd, rs1, rs2	divide unsigned	rd = rs1 / rs2
0110011 (51)	110	0000001	R	rem rd, rs1, rs2	remainder (signed)	rd = rs1 % rs2
0110011 (51)	111	0000001	R	remu rd, rs1, rs2	remainder unsigned	rd = rs1 % rs2

Table B.6 RVC: RISC-V compressed (16-bit) instructions

op	instr _{15:10}	funct2	类型	RVC 指令 32位等效	00 (0) 000----
00 (0)	001---	-	CL	c.addi4spn rd', sp, imm	addi rd', sp, ZeroExt(imm)*4
00 (0)	010---	-	CL	c.fld fd', imm(rs1')	fld fd', (ZeroExt(imm)*8)(rs1')
00 (0)	011---	-	CL	c.lw rd', imm(rs1')	lw rd', (ZeroExt(imm)*4)(rs1')
00 (0)	101---	-	CL	c.flw fd', imm(rs1')	flw fd', (ZeroExt(imm)*4)(rs1')
00 (0)	110---	-	CS	c.fsd fs2', imm(rs1')	fsd fs2', (ZeroExt(imm)*8)(rs1')
00 (0)	111---	-	CS	c.sw rs2', imm(rs1')	sw rs2', (ZeroExt(imm)*4)(rs1')
00 (0)	111---	-	CS	c.fsw fs2', imm(rs1')	fsw fs2', (ZeroExt(imm)*4)(rs1')
01 (1)	000000	-	CI	c.nop (rs1=0,imm=0)	nop
01 (1)	000---	-	CI	c.addi rd, imm	addi rd, rd, SignExt(imm)
01 (1)	001---	-	CJ	c.jal label	jal ra, label
01 (1)	010---	-	CI	c.li rd, imm	addi rd, x0, SignExt(imm)
01 (1)	011---	-	CI	c.lui rd, imm	lui rd, {14{imm ₅ }, imm}
01 (1)	011---	-	CI	c.addi16sp sp, imm	addi sp, sp, SignExt(imm)*16
01 (1)	100-00	-	CB'	c.srli rd', imm	srli rd', rd', imm
01 (1)	100-01	-	CB'	c.srai rd', imm	srai rd', rd', imm
01 (1)	100-10	-	CB'	c.andi rd', imm	andi rd', rd', SignExt(imm)
01 (1)	100011	00	CS'	c.sub rd', rs2'	sub rd', rd', rs2'
01 (1)	100011	01	CS'	c.xor rd', rs2'	xor rd', rd', rs2'
01 (1)	100011	10	CS'	c.or rd', rs2'	or rd', rd', rs2'
01 (1)	100011	11	CS'	c.and rd', rs2'	and rd', rd', rs2'
01 (1)	101---	-	CJ	c.j label	jal x0, label
01 (1)	110---	-	CB	c.beqz rs1', label	beq rs1', x0, label
01 (1)	111---	-	CB	c.bnez rs1', label	bne rs1', x0, label
10 (2)	000---	-	CI	c.slli rd, imm	slli rd, rd, imm
10 (2)	001---	-	CI	c.fldsp fd, imm	fld fd, (ZeroExt(imm)*8)(sp)
10 (2)	010---	-	CI	c.lwsp rd, imm	lw rd, (ZeroExt(imm)*4)(sp)
10 (2)	011---	-	CI	c.flwsp fd, imm	flw fd, (ZeroExt(imm)*4)(sp)
10 (2)	1000--	-	CR	c.jr rs1 (rs1≠0,rs2=0)	jalr x0, rs1, 0
10 (2)	1000--	-	CR	c.mv rd, rs2 (rd≠0,rs2≠0)	add rd, x0, rs2
10 (2)	1001--	-	CR	c.ebreak (rs1=0,rs2=0)	ebreak
10 (2)	1001--	-	CR	c.jalr rs1 (rs1≠0,rs2=0)	jalr ra, rs1, 0
10 (2)	1001--	-	CR	c.add rd, rs2 (rs1≠0,rs2≠0)	add rd, rd, rs2
10 (2)	101---	-	CSS	c.fsdsp fs2, imm	fsd fs2, (ZeroExt(imm)*8)(sp)
10 (2)	110---	-	CSS	c.swsp rs2, imm	sw rs2, (ZeroExt(imm)*4)(sp)
10 (2)	111---	-	CSS	c.fswsp fs2, imm	fsw fs2, (ZeroExt(imm)*4)(sp)

rs1', rs2', rd': 3-bit register designator for registers 8-15: 000₂ = x8 or f8, 001₂ = x9 or f9, etc.

表 B.7 RISC-V 伪指令 伪指令ISC-V 指令 描述 操作 nop ad

di x0, x			
	0, 0	no operation	
li rd, imm _{11:0}	addi rd, x0, imm _{11:0}	load 12-bit immediate	rd = SignExtend(imm _{11:0})
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called “register copy”)	rd = rs1
not rd, rs1	xori rd, rs1, -1	one’s complement	rd = ~rs1
neg rd, rs1	sub rd, x0, rs1	two’s complement	rd = -rs1
seqz rd, rs1	sltiu rd, rs1, 1	set if = 0	rd = (rs1 == 0)
snez rd, rs1	sltu rd, x0, rs1	set if ≠ 0	rd = (rs1 ≠ 0)
sltz rd, rs1	slt rd, rs1, x0	set if < 0	rd = (rs1 < 0)
sgtz rd, rs1	slt rd, x0, rs1	set if > 0	rd = (rs1 > 0)
beqz rs1, label	beq rs1, x0, label	branch if = 0	if (rs1 == 0) PC = label
bnez rs1, label	bne rs1, x0, label	branch if ≠ 0	if (rs1 ≠ 0) PC = label
blez rs1, label	bge x0, rs1, label	branch if ≤ 0	if (rs1 ≤ 0) PC = label
bgez rs1, label	bge rs1, x0, label	branch if ≥ 0	if (rs1 ≥ 0) PC = label
bltz rs1, label	blt rs1, x0, label	branch if < 0	if (rs1 < 0) PC = label
bgtz rs1, label	blt x0, rs1, label	branch if > 0	if (rs1 > 0) PC = label
ble rs1, rs2, label	bge rs2, rs1, label	branch if ≤	if (rs1 ≤ rs2) PC = label
bgt rs1, rs2, label	blt rs2, rs1, label	branch if >	if (rs1 > rs2) PC = label
bleu rs1, rs2, label	bgeu rs2, rs1, label	branch if ≤ (unsigned)	if (rs1 ≤ rs2) PC = label
bgtu rs1, rs2, label	bltu rs2, rs1, offset	branch if > (unsigned)	if (rs1 > rs2) PC = label
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol
l{b h w} rd, symbol	auipc rd, symbol _{31:12} l{b h w} rd, symbol _{11:0} (rd)	load global variable	rd = [PC + symbol]
s{b h w} rs2, symbol, rs1	auipc rs1, symbol _{31:12} s{b h w} rs2, symbol _{11:0} (rs1)	store global variable	[PC + symbol] = rs2
crr rd, csr	crrs rd, csr, x0	read CSR	rd = csr
crrw csr, rs1	crrw x0, csr, rs1	write CSR	csr = rs1

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a 压伯 and a global variable, respectively.

表 B.8 特权/CSR 指令

操作码	funct3	类型	指令	描述	操作	
1110011 (115)	000	I	ecall		将控制权转交给操作系统 (imm=0)	
1110011 (115)	000	I	ebreak		transfer control to debugger (imm=1)	
1110011 (115)	000	I	uret		return from user exception (rs1=0,rd=0,imm=2)	PC = uepc
1110011 (115)	000	I	sret		return from supervisor exception (rs1=0,rd=0,imm=258)	PC = sepc
1110011 (115)	000	I	mret		return from machine exception (rs1=0,rd=0,imm=770)	PC = mepc
1110011 (115)	001	I	crrw rd,csr,rs1		CSR read/write (imm=CSR number)	rd = csr, csr = rs1
1110011 (115)	010	I	crrs rd,csr,rs1		CSR read/set (imm=CSR number)	rd = csr, csr = csr rs1
1110011 (115)	011	I	crrc rd,csr,rs1		CSR read/clear (imm=CSR number)	rd = csr, csr = csr & ~rs1
1110011 (115)	101	I	crrwi rd,csr,uimm		CSR read/write immediate (imm=CSR number)	rd = csr, csr = ZeroExt(uimm)
1110011 (115)	110	I	crrsi rd,csr,uimm		CSR read/set immediate (imm=CSR number)	rd = csr, csr = csr ZeroExt(uimm)
1110011 (115)	111	I	crrci rd,csr,uimm		CSR read/clear immediate (imm=CSR number)	rd = csr, csr = csr & ~ZeroExt(uimm)

For privileged / CSR instructions, the 5-bit unsigned immediate, uimm, is encoded in the rs1 field.

赞美 *Digital Design and Computer Architecture*

RISC-V 版本

Harris和Harris展示了如何从门级电路一直设计到微架构层面的RISC-V处理器。他们清晰的讲解结合全面的方法，完整呈现了数字设计与RISC-V架构的全貌。借助学生在现代FPGA上运行大型数字设计的绝佳机会，这种方法既富有启发性又具有教育意义。

David A. Patterson 加州大学伯克利分校

哈里斯与哈里斯将如此广阔的领域浓缩为一本书！随着半导体制造日趋成熟，数字设计与计算机体系结构的重要性只会与日俱增。读者将获得对这两个主题通俗易懂且全面的阐述，并清晰理解RISC-V指令集架构。

Andrew Waterman SiFive

已有优秀的数字设计教材和计算机硬件组织教材——而本书两者兼备！其独特之处在于能够融会贯通。通过从基础出发，本书让RISC-V架构变得易于理解，且我发现其中的习题在多门课程中都是极佳的资源。

罗伊·克拉维茨 波特兰州立大学

2008年首次阅读Harris与Harris合著的MIPS教材时，我便认为这是计算机体系结构领域最优秀的教学书籍之一，并立即将其引入课堂。十三年后，有幸审阅这本全新的RISC-V版本，我对该系列著作的评价始终如一：*Digital Design and Computer Architecture: RISC-V Edition* 堪称卓越之作——内容清晰透彻、体系完备，兼具极高的教学价值，完全契合我们在数字设计与计算机体系结构领域的课程设置。期待将这本RISC-V教材应用于我的课堂教学。

Daniel Chaver Martinez University Complutense of Madrid

数字设计与计算机体系结构

RISC-V 版本

数字设计与计算机体系 结构

RISC-V 版本

莎拉·L·哈里斯
大卫·哈里斯



ELSEVIER

AMSTERDAM • BOSTON • HEIDELBERG • LONDON
NEW YORK • OXFORD • PARIS • SAN DIEGO
SAN FRANCISCO • SINGAPORE • SYDNEY • TOKYO

Morgan Kaufmann is an imprint of Elsevier

MK
MORGAN KAUFMANN

摩根·考夫曼是爱思唯尔的印记，地址位于美国马萨诸塞州剑桥市汉普郡街50号5楼，邮编02139

版权所有 © 2022 Elsevier Inc. 保留所有权利。

未经出版商书面许可，不得以任何形式或任何方式（包括电子或机械方式，如影印、录制或任何信息存储与检索系统）复制或传播本出版物的任何部分。有关如何获取许可、出版商许可政策的更多信息，以及我们与版权清算中心和版权许可机构等组织的安排，请访问我们的网站：www.elsevier.com/permissions。

本书及其中的各项内容均受出版商版权保护（除非本文另有注明）。

通知

本领域的知识和最佳实践在不断变化。随着新的研究和经验拓宽我们的理解，研究方法、专业实践或医疗治疗可能需要进行调整。

从业者和研究人员在评估和使用本文所述的任何信息、方法、化合物或实验时，必须始终依靠自身的经验和知识。在使用此类信息或方法时，他们应注意自身及他人的安全，包括对其负有专业责任的相关方。

在法律允许的最大范围内，出版商、作者、撰稿人或编辑均不对因产品责任、疏忽或其他原因造成的人身或财产伤害或损害，或对使用或操作本文材料中包含的任何方法、产品、说明或理念承担任何责任。

不列颠图书馆出版编目数据

本书的目录记录可从大英图书馆获取。

美国国会图书馆出版编目数据 本书的编目记录可从美国国会图书馆获取。

ISBN: 978-0-12-820064-3

For Information on all Morgan Kaufmann publications visit our website at <https://www.elsevier.com/books-and-journals>

Publisher: Katey Birtcher *Senior Acquisitions Editor:* Steve Merken *Editorial Project Manager:* Ruby Gammell/
Andrae Akeh Senior Project Manager: Manikandan Chandrasekaran *Cover Designer:* Victoria Pearson Esser

Typeset by MPS Limited, Chennai, India

Printed in the United States of America

最后一位数字是印刷编号：9 8 7 6 5 4 3 2 1



Working together
to grow libraries in
developing countries

www.elsevier.com • www.bookaid.org

目录

前言	xix
特性	x
x 在线补充材料	xxi
如何在课程中使用软件工具	
xxii 实验	
... xxiii RVfpga	
... xxiii 错误	
... xxiv 致谢	
... xxiv	
关于作者	xxv
第一章 从零到一	1
1.1 游戏计划	1
1.2 管理复杂性的艺术	2
1.2.1 <i>Abstraction</i>	
1.2.2 <i>Discipline</i>	
1.2.3 <i>The Three -Y's</i>	
4 1.3 数字抽象	
5 1.4 数制	
7 1.4.1 <i>Decimal Numbers</i>	
7 1.4.2 <i>Binary Numbers</i>	7
3 1.4.3 <i>Hexadecimal Numbers</i>	9
11 1.4.4 <i>Bytes, Nibbles, and All That Jazz</i>	11
12 1.4.5 <i>Binary Addition</i>	12
13 1.4.6 <i>Signed Binary Numbers</i>	13
1.5 逻辑门	17
1.5.1 <i>NOT Gate</i>	1
8 1.5.2 <i>Buffer</i>	18
3 1.5.3 <i>AND Gate</i>	18
19 1.5.4 <i>OR Gate</i>	19
19 1.5.5 <i>Other Two-Input Gates</i>	19
19 1.5.6 <i>Multiple-Input Gates</i>	19
20 1.6 数字抽象之下	20
20 1.6.1 <i>Supply Voltage</i>	
20 1.6.2 <i>Logic Levels</i>	
20 1.6.3 <i>Noise Margins</i>	
21	21

1.6.4 DC Transfer Characteristics	22
1.6.5 The Static Discipline	22
MOS晶体管	24
Semiconductors	25
Diodes	25
Capacitors	26
nMOS and pMOS Transistors	26
..... 26 1.7.5 CMOS NOT Gate	29
.. 29 1.7.6 Other CMOS Logic Gates	29
1.7.7 Transmission Gates	31
Pseudo-nMOS Logic	31
..... 32 1.8 功耗	32
..... 32 1.9 总结与展望	32
..... 34 习题	34
..... 36 面试题	36
..... 50	50
 章节 2 组合逻辑设计	53
2.1 引言	53
布尔方程	56
Terminology	56
Sum-of-Products Form	56
56 2.2.3 Product-of-Sums Form	58
3 布尔代数	58
Axioms	59
Theorems of One Variable	59
..... 59 2.3.3 Theorems of Several Variables	60
..... 60 2.3.4 The Truth Behind It All	62
..... 62 2.3.5 Simplifying Equations	63
..... 63 2.4 从逻辑到门电路	64
..... 64 2.5 多级组合逻辑	67
..... 67 2.5.1 Hardware Reduction	68
..... 68 2.5.2 Bubble Pushing	69
..... 69 2.6 X与Z, 哦天哪	71
..... 71 2.6.1 Illegal Value: X	71
..... 71 2.6.2 Floating Value: Z	72
..... 72 2.7 卡诺图	73
..... 73 2.7.1 Circular Thinking	74
..... 74 2.7.2	75
Logic Minimization with K-Maps	75
..... 75 2.7.3 Don't Cares	79
..... 79 2.7.4 The Big Picture	80
..... 80 2.8 组合构建模块	81
..... 81 2.8.1 Multiplexers	81
..... 81 2.8.2 Decoders	84
..... 84	84

第四章 硬件描述语言	171
4.1 引言	171 4.1.1
Modules	171 4.1.2
Language Origins	
4.4.1 Registers	191
4.4.2 Resettable Registers	192
4.4.3 Enabled Registers	194
4.4.4 Multiple Registers	195
4.4.5 Latches	196
4.5 更多组合逻辑	196
4.5.1 Case Statements	199
4.5.2 If Statements	200
4.5.3 Truth Tables with Don't Cares	203
4.5.4 Blocking and Nonblocking Assignments	203
4.6 有限状态机	207
4.7 数据类型	211 4.7.1
SystemVerilog	212 4.7.2 VHDL
4.8 参数化模块	213
4.9 测试平台	215
4.10 总结	218
习题	222
面试题	224
	235
第五章 数字构建模块	237
5.1 引言	237
5.2 算术电路	237 5.2.1 Addition
	237 5.2.2 Subtraction
	244

6.4 机器语言	332
6.4.1 R-Type Instructions	332
6.4.2 I-Type Instructions	334
6.4.3 S/B-Type Instructions	336
6.4.4 U/J-Type Instructions	338
6.4.5 Immediate Encodings	340
6.4.6 Addressing Modes	341
6.4.7 Interpreting Machine Language Code	
6.5 灯光、摄影、开拍：编译、汇编与加载	
344 6.5.1 The Memory Map	
344 6.5.2 Assembler Directives	
346 6.5.3 Compiling	
348 6.5.4 Assembling	350
5 Linking	353
Loading	355
6.6 零散内容	355
6.6.1 Endianness	355
6.6.2 Exceptions	356
6.6.3 Signed and Unsigned Instructions	360
6.6.4 Floating-Point Instructions	361
6.6.5 Compressed Instructions	363
6.7 RISC-V 架构的演进	363
6.7.1 RISC-V Base Instruction Sets and Extensions	364
6.7.2 Comparison of RISC-V and MIPS Architectures	365
6.7.3 Comparison of RISC-V and ARM Architectures	365
6.8 另一种视角：x86 架构	366
6.8.1 x86 Registers	366
6.8.2 x86 Operands	367
6.8.3 Status Flags	369
6.8.4 x86 Instructions	369
6.8.5 x86 Instruction Encoding	371
6.8.6 Other x86 Peculiarities	372
6.8.7 The Big Picture	373
6.9 总结	374
375 面试问题	390
第7章 微架构	393
7.1 引言	393
7.1.1 Architectural State and Instruction Set	393
7.1.2 Design Process	394
7.1.3 Microarchitectures	396

7.2 性能分析	397
7.3 单周期处理器	398
7.3.1 <i>Sample Program</i>	399
7.3.2 <i>Single-Cycle Datapath</i>	399
7.3.3 <i>Single-Cycle Control</i>	407
7.3.4 <i>More Instructions</i>	410
7.3.5 <i>Performance Analysis</i>	412
7.4 多周期处理器	415
7.4.1 <i>Multicycle Datapath</i>	416
7.4.2 <i>Multicycle Control</i>	422
7.4.3 <i>More Instructions</i>	432
7.4.4 <i>Performance Analysis</i>	435
7.5 流水线处理器	439
7.5.1 <i>Pipelined Datapath</i>	441
7.5.2 <i>Pipelined Control</i>	443
7.5.3 <i>Hazards</i>	443
7.5.4 <i>Performance Analysis</i>	454
7.6 HDL表示	456
7.6.1 <i>Single-Cycle Processor</i>	457
7.6.2 <i>Generic Building Blocks</i>	461
7.6.3 <i>Testbench</i>	464
7.7 高级微架构	468
7.7.1 <i>Deep Pipelines</i>	468
7.7.2 <i>Micro-Operations</i>	469
7.7.3 <i>Branch Prediction</i>	470
7.7.4 <i>Superscalar Processors</i>	472
7.7.5 <i>Out-of-Order Processor</i>	473
7.7.6 <i>Register Renaming</i>	476
7.7.7 <i>Multithreading</i>	478
7.7.8 <i>Multiprocessors</i>	479
7.8 现实视角: RISC-V微架构的演进	482
7.9 总结	486
习题	488
面试题	497
第八章 记忆系统	499
8.1 引言	499
8.2 内存系统性能分析	503
8.3 缓存	505
8.3.1 <i>What Data is Held in the Cache?</i>	505
8.3.2 <i>How is Data Found?</i>	506
8.3.3 <i>What Data is Replaced?</i>	514
8.3.4 <i>Advanced Cache Design</i>	515

8.4 虚拟内存	519
8.4.1 <i>Address Translation</i>	522
8.4.2 <i>The Page Table</i>	523
8.4.3 <i>The Translation Lookaside Buffer</i>	525
8.4.4 <i>Memory Protection</i>	526
8.4.5 <i>Replacement Policies</i>	527
8.4.6 <i>Multilevel Page Tables</i>	527
8.5 总结	530
后记	530
练习	532
面试问题	541
第9章 嵌入式I/O系统	542
9.1 引言	542
第9章作为在线补充材料提供	542.e1
9.1 引言	542.e1
9.2 内存映射I/O	542.e1
9.3 嵌入式I/O系统	542.e3
9.3.1 <i>RED-V Board</i>	542.e3
9.3.2 <i>FE310-G002 System-on-Chip</i>	542.e5
9.3.3 <i>General-Purpose Digital I/O</i>	542.e5
9.3.4 <i>Device Drivers</i>	542.e10
9.3.5 <i>Serial I/O</i>	542.e14
9.3.6 <i>Timers</i>	542.e29
9.3.7 <i>Analog I/O</i>	542.e30
9.3.8 <i>Interrupts</i>	542.e39
9.4 其他微控制器外设	542.e43
9.4.1 <i>Character LCDs</i>	542.e43
9.4.2 <i>VGA Monitor</i>	542.e45
9.4.3 <i>Bluetooth Wireless Communication</i>	542.e53
9.4.4 <i>Motor Control</i>	542.e54
9.5 总结	542.e64
附录A 数字系统实现	543
A.1 引言	543
附录A作为在线补充材料提供	543.e1
A.1 引言	543.e1
A.2 逻辑	543.e1
A.2.1 <i>Logic Gates</i>	543.e2
A.2.2 <i>Other Functions</i>	543.e2

A.3 可编程逻辑	543.e2 A.3.1
PROMs	543.e2 A.3.2 PLAs
.....	543.e6 A.3.3 FPGAs ...
.....	543.e7 A.4 专用集成电路
.....	543.e9 A.5 数据手册
.....	543.e9 A.6 逻辑系列
.....	543.e15 A.7 开关与发光二极管
.....	543.e17 A.7.1 <i>Switches</i>
.....	543.e17 A.7.2 <i>LEDs</i>
.....	543.e18 A.8 封装与组装
..	543.e19 A.8.1 <i>Packages</i>
e19	A.8.2 <i>Breadboards</i>
e20	A.8.3 <i>Printed Circuit Boards</i>
.....	543.e20 A.8.4 <i>Putting It All Together</i>
.....	543.e23 A.9 传输线
.....	543.e23
A.9.1 <i>Matched Termination</i>	543.e24
A.9.2 <i>Open Termination</i>	543.e26
A.9.3 <i>Short Termination</i>	543.e27
A.9.4 <i>Mismatched Termination</i>	543.e27
A.9.5 <i>When to Use Transmission Line Models</i>	
.....	543.e30 A.9.6
<i>Proper Transmission Line Terminations</i>	543.e30 A.9.7
<i>Derivation of Z_0</i>	543.e32 A.9.8
<i>Derivation of the Reflection Coefficient.</i>	543.e33 A.9.9
<i>Putting It All Together</i>	543.e34
A.10 经济学	543.e35
附录 B RISC-V 指令集概述	544
附录C C程序设计	545
C.1 引言	545
附录C作为在线补充材料提供.....	545.e1
C.1 引言	545.e1 C.2 欢迎
欢迎来到C	545.e3 C.2.1
<i>C Program Dissection</i>	545.e3 C.2.2
<i>Running a C Program</i>	545.e4 C.3 编译 ...
.....	545.e5 C.3.1 <i>Comments</i> ...
.....	545.e5 C.3.2 <i>#define</i>
.....	545.e5 C.3.3 <i>#include</i>
.....	545.e6

C.4 变量	545.e7	C.4
.1 <i>Primitive Data Types</i>	545.e8	C.4.2
<i>Global and Local Variables</i>	545.e9	C.4.3
<i>Initializing Variables</i>	545.e11	C.5 运算符 ..
.....	545.e11	C.6 函数调用 ...
.....	545.e15	C.7 控制流语句
.....	545.e16	C.7.1 <i>Conditional Statements</i>
.....	545.e17	C.7.2 <i>Loops</i>
.....	545.e19	C.8 更多数据类型
.....	545.e21	
C.8.1 <i>Pointers</i>	545.e21	
C.8.2 <i>Arrays</i>	545.e2	
3 C.8.3 <i>Characters</i>	545.e	
27 C.8.4 <i>Strings</i>	545.e	
.e27 C.8.5 <i>Structures</i>	54	
5.e29 C.8.6 <i>typedef</i>	5	
45.e31 C.8.7 <i>Dynamic Memory Allocation</i>	54	
5.e32 C.8.8 <i>Linked Lists</i>	545	
.e33		
C.9 标准库	545.e35	
C.9.1 <i>stdio</i>	545.e35	
C.9.2 <i>stdlib</i>	545.e40	
C.9.3 <i>math</i>	545.e42	
C.9.4 <i>string</i>	545.e42	
C.10 编译器和命令行选项	545.e43	
C.10.1 <i>Compiling Multiple C Source Files</i>	545.e43	
C.10.2 <i>Compiler Options</i>	545.e43	
C.10.3 <i>Command Line Arguments</i>	545.e44	
C.11 常见错误	545.e44	
延伸阅读	547	
索引	549	

前言

本书的独特之处在于，它从计算机体系结构的角度阐述数字逻辑设计，从最初的1和0开始，一直引导到微处理器的设计。

我们认为，构建微处理器是工程与计算机科学学生必经的一项特殊仪式。对于初学者而言，处理器的内部运作看似近乎神奇，但经过细致讲解后便会发现其原理其实直截了当。数字设计本身便是一门强大而令人兴奋的学科。汇编语言编程揭示了处理器内部使用的语言。微架构则是将这一切融会贯通的关键纽带。

这本日益流行的教材前两版涵盖了MIPS和ARM架构。作为最早的精简指令集计算架构之一，MIPS简洁且极其易于理解和构建。MIPS至今仍是一套重要的架构，因为它启发了包括RISC-V在内的许多后续架构。ARM架构凭借其高效性和丰富的生态系统，在过去几十年中人气激增。已有超过500亿颗ARM处理器出货，全球超过75%的人口在使用搭载ARM处理器的产品。

过去十年中，RISC-V在教学和商业领域逐渐成为日益重要的架构。作为首个广泛使用的开源计算机架构，RISC-V兼具MIPS的简洁性与现代处理器的灵活性和特性。

从教学角度来看，MIPS、ARM和RISC-V版本的学习目标是一致的。RISC-V架构具有可扩展性和压缩指令等特性，这些特性有助于提高效率，但也增加了一定的复杂性。这三种微架构也相似，其中MIPS和RISC-V架构有许多共同之处。只要市场有需求，我们将继续提供MIPS、ARM和RISC-V版本。

特征

SystemVerilog与VHDL的并排覆盖

硬件描述语言（HDL）是现代数字设计实践的核心。然而，设计人员在使用两种主流语言——SystemVerilog和VHDL——时各占一半。本书在第4章介绍组合逻辑和时序逻辑设计后，立即引入HDL。随后在第5章和第7章中，利用HDL设计更大的构建模块及完整的处理器。不过，第4章可以跳过，后续章节对于选择不涉及HDL的课程仍然适用。

本书的独特之处在于将SystemVerilog和VHDL并排呈现，使读者能够同时学习这两种语言。第4章描述了适用于两种HDL的原则，随后在相邻列中提供了特定语言的语法和示例。这种并排处理方式便于教师选择任意一种HDL，也便于读者在课堂或专业实践中从一种语言过渡到另一种语言。

rISC-V架构与微架构

第6章和第7章深入介绍了RISC-V架构和微架构。RISC-V是一种理想的架构，因为它是一个真实的架构，已应用于越来越多的商业产品中，同时它简洁明了且易于学习。此外，由于其在商业和爱好者领域的普及，RISC-V架构拥有现成的仿真和开发工具。

现实世界视角

除了在讨论RISC-V架构时提供实际视角外，第6章还展示了Intel x86处理器的架构以提供另一种视角。第9章（作为在线补充材料提供）还以SparkFun的RED-V RedBoard为背景描述了外设，这是一款以SiFive的Freedom E310 RISC-V处理器为核心的流行开发板。这些实际视角章节展示了各章中的概念如何与许多个人电脑和消费电子产品中的芯片相关联。

可访问的高级微架构概述

第7章概述了现代高性能微架构特性，包括分支预测、超标量、乱序执行、多线程以及多核处理器。

该处理方法适用于初学课程的学生，并展示了书中的微架构如何扩展到现代处理器。

章末习题与面试问题

学习数字设计的最佳方式就是动手实践。每章末尾都配有大量练习，供读者巩固所学内容。练习之后是一系列面试题，这些题目来自我们的行业同事，他们曾向申请该领域工作的学生提问。这些问题有助于求职者初步了解面试过程中通常会遇到的典型问题。练习答案可通过本书的配套网站和教师网站获取。

在线补充材料

补充材料可在 ddcabook.com 或出版商网站在线获取：<https://www.elsevier.com/books-and-journals/book-companion/9780128200643>。这些配套网站（对所有读者开放）包含以下内容：

视频讲座链接 ▶ 奇数编号习题解答 ▶ 文本中的图表（PDF和PPTX格式） ▶ 来自英特尔的专业级计算机辅助设计（CAD）工具链接[®] 如何使用PlatformIO（Visual Studio Code的扩展）编译、汇编和模拟RISC-V处理器的C语言及汇编代码的说明 ▶ RISC-V处理器的硬件描述语言（HDL）代码 ▶ 英特尔的Quartus实用提示 ▶ PowerPoint（PPTX）格式的讲座幻灯片 ▶ 示例课程与实验材料 ▶ 勘误表

教师站点（注册于<https://inspectioncopy.elsevier.com>的教师可访问）包含以下内容：

所有习题解答 ▶ 实验解答

edX 慕课

本书还配套了一门通过EdX平台开设的大规模开放在线课程（MOOC）。该课程包含视频讲座、互动练习

问题、交互式问题集和实验。该MOOC分为两部分：数字设计（ENGR 85A）和计算机体系结构（ENGR 85B），由HarveyMuddX提供（在EdX上搜索“Digital Design HarveyMuddX”和“Computer Architecture HarveyMuddX”）。EdX不收取视频访问费用，但会对交互式练习和证书收费。EdX为有经济需求的学生提供折扣。

如何在课程中使用软件工具

英特尔Quartus软件

Quartus软件（Web版或Lite版）是英特尔专业级Quartus™ FPGA设计工具的免费版本。它允许学生以原理图形式或使用SystemVerilog、VHDL硬件描述语言（HDL）输入数字设计。完成设计输入后，学生可使用随英特尔Quartus软件提供的ModelSim™-Intel FPGA版或入门版仿真电路。Quartus还内置了支持SystemVerilog和VHDL的逻辑综合工具。

Web版或精简版与专业版的区别在于，Web版或精简版仅支持部分最常见的Altera FPGA。免费版ModelSim在仿真超过10,000行HDL代码时会降低性能，而专业版ModelSim则不会。

PlatformIO

PlatformIO是Visual Studio Code的一个扩展，可作为RISC-V的软件开发工具包（SDK）。随着每个新平台对应的SDK激增，PlatformIO通过为大量平台和设备提供统一接口，简化了编程和使用各种处理器的流程。该工具可免费下载，并能与SparkFun的RED-V RedBoard配合使用，如配套网站提供的实验所述。PlatformIO提供了对商业RISC-V编译器的访问权限，使学生能够编写C语言和汇编程序，进行编译，然后在SparkFun的RED-V RedBoard上运行和调试这些程序（参见第9章及配套实验）。

金星rISc-V汇编模拟器

金星模拟器（访问地址：<https://www.kvakil.me/venus/>）是一款基于网页的RISC-V汇编模拟器。程序在“编辑器”选项卡中编写（或复制/粘贴），随后在“模拟器”选项卡中进行模拟和运行。程序运行时，可查看寄存器与内存内容。

LaBS

配套网站包含一系列实验的链接，内容涵盖从数字设计到计算机架构的主题。这些实验教授学生如何使用Quartus工具进行设计输入、仿真、综合和实现。实验还包括使用PlatformIO和SparkFun的RED-V RedBoard进行C语言和汇编语言编程的主题。

综合设计完成后，学生可使用Altera DE2、DE2-115、DE0或其他FPGA开发板实现其设计。实验指导书以DE2或DE-115开发板为目标平台。这些性能强劲且价格极具竞争力的开发板可通过de2-115.terasic.com获取。开发板搭载的FPGA可通过编程实现学生设计方案。我们提供的实验指导详细说明了如何使用Quartus软件在DE2-115开发板上实现精选设计方案。

。

为完成实验，学生需下载并安装英特尔Quartus Web或Lite版以及带有PlatformIO扩展的Visual Studio Code。教师也可选择在实验室机器上安装这些工具。实验指南包含如何在DE2/DE2-115开发板上实现项目的说明。实现步骤可跳过，但我们发现其极具价值。我们已在Windows系统上测试过这些实验，但相关工具也适用于Linux系统。

rVfpga

RISC-V FPGA（简称RVfpga）是一套免费的两门课程系列，可在学习本书内容后完成。第一门课程展示如何将商业RISC-V内核部署到FPGA上，使用RISC-V汇编或C语言进行编程，为其添加外设，并分析和修改内核及存储系统，包括向内核添加指令。该课程使用基于西部数据开源商业SweRV EH1内核（<https://www.westerndigital.com/company/innovations/risc-v>）的开源SweRVolf片上系统（SoC）（<https://github.com/chipsalliance/Cores-SweRVolf>）。课程还介绍如何使用开源HDL仿真器Verilator以及西部数据的开源RISC-V指令集仿真器（ISS）Whisper。第二门课程RVfpga-SoC展示如何使用SweRV EH1内核、互连和存储器等构建模块，基于SweRVolf构建SoC。随后，课程指导用户在RISC-V SoC上加载并运行Zephyr操作系统。所有必要的软件和系统源代码（Verilog/SystemVerilog文件）均免费提供，且课程可通过仿真完成，因此无需硬件。RVfpga材料可通过Imagination Technologies大学计划免费注册获取：<https://university.imgtec.com/rvfpga/>。

错误

所有经验丰富的程序员都知道，任何具有一定复杂性的程序都不可避免地存在错误。书籍也是如此。我们已竭尽全力查找并修正本书中的错误。然而，一些错误无疑仍然存在。我们将在本书的网页上维护一份勘误列表。

请将您的错误报告发送至 ddcbugs@gmail.com。首位报告实质性错误并提供修复方案、且该方案被我们用于未来印刷版本的人，将获得1美元奖金！

致谢

我们感谢Steve Merken、Nate McFadden、Ruby Gammell、Andrae Akeh、Manikandan Chandrasekaran以及Morgan Kaufmann团队的其他成员为本书的出版所付出的辛勤努力。我们喜爱Duane Bibby的艺术作品，他的漫画为各章增添了生动色彩。

我们感谢马修·沃特金斯（Matthew Watkins）贡献了第7章中关于异构多处理器的部分，以及乔什·布雷克（Josh Brake）为第9章嵌入式I/O系统所做的贡献。我们非常感谢马特奥·马尔科维奇（Mateo Markovic）和乔迪·赖德（Geordie Ryder）审阅本书并参与习题解答的工作。众多其他审阅者也对本书的改进做出了重要贡献，其中包括丹尼尔·查弗·马丁内斯（Daniel Chaver Martinez）、罗伊·克拉维茨（Roy Kravitz）、兹沃尼米尔·班迪奇（Zvonimir Bandic）、朱塞佩·迪·卢纳（Giuseppe Di Luna）、斯特芬·保罗（Steffen Paul）、拉维·米塔尔（Ravi Mittal）、詹妮弗·维尼库斯（Jennifer Winikus）、赫沙姆·奥姆兰（Hesham Omran）、安赫尔·索利斯（Angel Solis）、赖纳·迪松（Reiner Dizon）和奥洛夫·金德格伦（Olof Kindgren）。我们还要感谢哈佛·穆德学院（Harvey Mudd College）和内华达大学拉斯维加斯分校（UNLV）课程中的学生们，他们为本教材的草稿提供了宝贵的反馈。最后但同样重要的是，我们两人都感谢各自家人的爱与支持。

关于作者

莎拉·L·哈里斯是内华达大学拉斯维加斯分校电气与计算机工程系的副教授。她在斯坦福大学获得电气工程博士学位和硕士学位。莎拉还曾在惠普、圣地亚哥超级计算机中心和英伟达工作过。她热爱教学、探索和开发新技术、旅行、弹奏吉他及其他多种乐器，以及陪伴家人。她最近的研究包括设计仿生假肢和在硬件中实现机器学习算法。

大卫·哈里斯是哈佛·穆德学院工程设计与副教授席哈维·S·穆德教授，并担任系副主任。他在斯坦福大学获得电气工程博士学位，在麻省理工学院获得电气工程与计算机科学工程硕士学位。在进入斯坦福大学之前，他曾在英特尔担任逻辑与电路设计师，参与安腾和奔腾II处理器的研发。此后，他为博通、太阳微系统、惠普、埃文斯与萨瑟兰及其他设计公司提供咨询。大卫热爱教学、芯片与飞机制造以及户外探索。工作之余，他通常会在徒步、飞行或与家人共度时光。大卫拥有约十几项专利，并撰写了三本关于芯片设计的教科书，以及七本南加州山脉的指南书。

